FYS5419 - Project 1

Insert Name

2025-03-20

Contents

1 Part a) 2

1.1 One-qubit operations . . . 2

1.1.1 Pauli gates . . . 3

1.1.2 Hadamard gate . . . 5

1.1.3 Phase shift-gate . . . 5

1.2 Bell states . . . 7

1.2.1 Binary Qubit Measurement . . . 7

2 Part b) One-Qubit Hamiltonian Eigenvalue Problem 9

3 Part c) VQE for One-Qubit Hamiltonian 12

4 Part d) Two-Qubit Hamiltonian Eigenvalue Problem 12

5 Appendix 14

5.1 Code Repository . . . 14

5.2 Source Code . . . 14

5.2.1 Qubit Class . . . 14

5.2.2 One-Qubit Hamiltonian Energies . . . 14

5.2.3 Two-Qubit Hamiltonian Energies and Entropies . . . 14

5.2.4 VQE Ansatz . . . 14

5.2.5 Parameter Shift Gradient . . . 14

5.2.6 VQE Gradient Descent (ADAM) . . . 14

5.2.7 One-Qubit Hamiltonian Quantum Circuit Computation . . . 14

1 Part a)

1.1 One-qubit operations

A single qubit state $|\psi\rangle \in \P\mathcal{H} \cong \mathbb{C}^2$ can be represented as a 2D complex vector

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle\,, \tag{1}$$

where $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$ are the polar and azimuthal angle in the Bloch sphere, respectively. The computational basis states $|0\rangle$ and $|1\rangle$ are given by

$$|0\rangle = |\uparrow_{\hat{\mathbf{z}}}\rangle := \begin{bmatrix}1\\0\end{bmatrix}, \quad |1\rangle = |\downarrow_{\hat{\mathbf{z}}}\rangle := \begin{bmatrix}0\\1\end{bmatrix}, \tag{2}$$

corresponding to spin-up and spin-down along the z-axis. Using the Numpy package in Python, a single qubit can be represented with the function:

```
import numpy as np
def create_qubit(theta: float, phi: float) -> np.ndarray:
  return np.array(
    [np.cos(theta / 2), np.exp(1j * phi) * np.sin(theta / 2)],
    dtype=np.complex128
  )
```

A more general Python implementation of n-ary qubit state is shown in Program 10

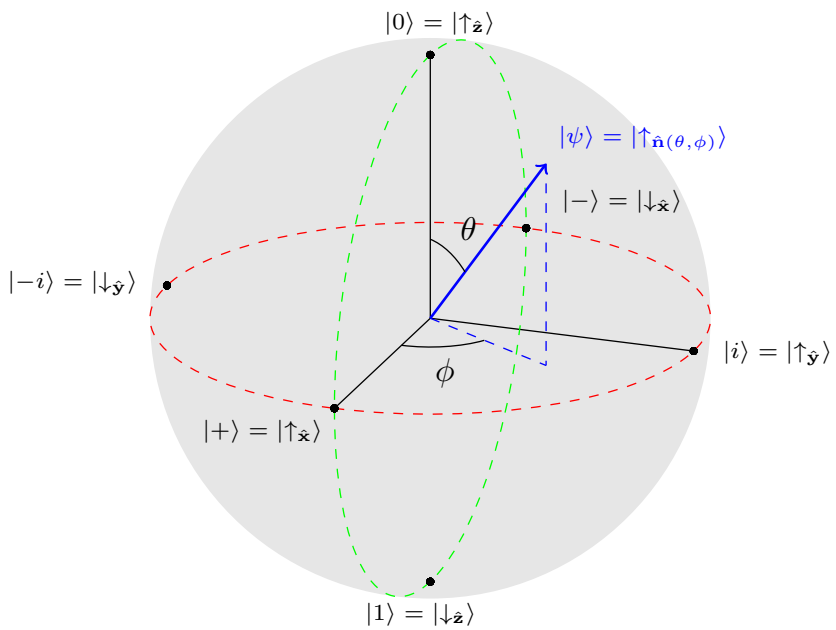

Figure 1: Bloch sphere representation of a qubit.

1.1.1 Pauli gates

In the standard basis $\{ \ |0\rangle, |1\rangle \ \}$, the Pauli gates are given by

$$\begin{aligned}
\hat{X} =& \hat{\sigma}_x = |0\rangle\langle 1| + \langle 1||0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \\
\hat{Y} =& \hat{\sigma}_y = -i\,|0\rangle\langle 1| + i\,|1\rangle|0\rangle = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \\
\hat{Z} =& \hat{\sigma}_x = |0\rangle\langle 0| - |1\rangle\langle 1| = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.
\end{aligned}$$

Geometrically, the Pauli gates generate half-turns about their respective axes. This can be understood in the terms of the rotation gate $\hat{R}_{\hat{\mathbf{n}}}(\theta)$, which rotates a qubit by an angle θ around a unit axis $\hat{\mathbf{n}} \in \mathbb{S}^2$ in $\mathbb{R}^3$:

$$\hat{R}_{\hat{\mathbf{n}}}(\theta) = \exp\left(-i\frac{\theta}{2}\hat{\mathbf{n}}\cdot\boldsymbol{\sigma}\right) = \cos\left(\frac{\theta}{2}\right)\hat{I}_2 - i\sin\left(\frac{\theta}{2}\right)\hat{\mathbf{n}}\cdot\boldsymbol{\sigma}, \tag{3}$$

where $\sigma = (\hat{\sigma}_x, \hat{\sigma}_y, \hat{\sigma}_z)$ is the Pauli vector, and $\hat{I}_2$ is the identity operator. In particular for $\theta = \pi$, we obtain

$$\hat{R}_{\hat{\mathbf{n}}}(\pi) = \cos\left(\frac{\pi}{2}\right)\hat{I}_2 - i\sin\left(\frac{\pi}{2}\right)\hat{\mathbf{n}}\cdot\boldsymbol{\sigma} = -i\hat{\mathbf{n}}\cdot\boldsymbol{\sigma}, \tag{4}$$

which shows that each Pauli gate (up to a global phase factor $-i$) is equivalent to a π-rotation about its respective axis, i.e.

$$\hat{R}_x(\pi) = -i\hat{X}, \quad \hat{R}_y(\pi) = -i\hat{Y}, \quad \hat{R}_z(\pi) = -i\hat{Z}, \tag{5}$$

Pauli X gate Applying the Pauli-X gate to a qubit $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ with $\alpha, \beta \in \mathbb{C}$ with $|\alpha|^2 + |\beta|^2 = 1$ gives

$$\begin{aligned}
\hat{X}(\alpha\,|0\rangle + \beta\,|1\rangle) =& (\,|0\rangle\langle 1| + |1\rangle\langle 0|\,)(\alpha\,|0\rangle + \beta\,|1\rangle\,) \\
=& \beta\,|1\rangle + \alpha\,|0\rangle\,,
\end{aligned}$$

meaning that it flips the qubit. In Python, the Pauli-X gate can be implemented in the following way using Numpy:

```
import numpy as np
def apply_x_gate(qubit: np.ndarray) -> np.ndarray:
  x_gate = np.array([[0., 1.], [1., 0.]], dtype=np.complex128)
  return qubit @ x_gate
```

```
# Example
qubit = create_qubit(theta=0, phi=0) # |0> state: [1., 0.]
qubit_x_gate = apply_x_gate(qubit) # Returns |1> state: [0., 1.]
```

Pauli Y gate Applying the Y gate to a qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ yields

$$\begin{aligned}\hat{Y}(\alpha|0\rangle + \beta|1\rangle) =& \big(-i|0\rangle\langle 1| + i|1\rangle\langle 0|\big)\big(\alpha|0\rangle + \beta|1\rangle\big)\\ =& i\big(\alpha|1\rangle - \beta|0\rangle\big),\end{aligned}$$

whichs shows that it flips the qubit and multiplies the phase by i. In Python, the Pauli Y-gate can be implemented in the following way using Numpy:

```
import numpy as np
def apply_y_gate(qubit: np.ndarray) -> np.ndarray:
  y_gate = np.array([[0., -1.j], [1.j, 0.]], dtype=np.complex128)
  return qubit @ y_gate

# Example
qubit = create_qubit(theta=0, phi=0) # |0> state: [1., 0.]
qubit_y_gate = apply_y_gate(qubit) # Returns i|1> state: [0.+0.j 0. -1.j]
```

Pauli Z gate Applying the Pauli-Z gate to a qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ yields

$$\begin{aligned}\hat{Z}(\alpha|0\rangle + \beta|1\rangle) =& \big(|0\rangle\langle 0| - |1\rangle\langle 1|\big)\big(\alpha|0\rangle + \beta|1\rangle\big)\\ =& \alpha|0\rangle - \beta|1\rangle = \alpha|0\rangle + e^{i\pi}\beta|1\rangle,\end{aligned}$$

showing that it multiplies the phase of $|1\rangle$ by -1. In particular, the Z gate flips the $\hat{X}$ basis $\{\ |+\rangle, |-\rangle\ \}$, where

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \tag{6}$$

In Python, the Pauli Z-gate can be implemented in the following way using Numpy:

```
import numpy as np
def apply_z_gate(qubit: np.ndarray) -> np.ndarray:
  z_gate = np.array([[1., 0.], [0., -1.]], dtype=np.complex128)
  return qubit @ z_gate

# Example
qubit = create_qubit(theta=np.pi/2, phi=0) # |+> state: [0.70710678, 0.70710678]
qubit_z_gate = apply_z_gate(qubit) # Returns | -> state: [0.70710678, -0.70710678]
```

1.1.2 Hadamard gate

The Hadamard gate is given by

$$\hat{H} := \frac{1}{\sqrt{2}}(\hat{X} + \hat{Z}) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{7}$$

Geometrically, it generates a half-turn about the diagonal $(x + z)$-axis on the Bloch sphere. This can be expressed in terms of the rotation operator $\hat{R}_{\hat{\mathbf{n}}}(\theta)$ about $\hat{\mathbf{n}} = \frac{1}{\sqrt{2}}(1, 0, 1)$, which corresponding to the diagonal $(x + z)$-axis:

$$\begin{aligned} \hat{R}_{\hat{\mathbf{n}}}(\theta) &= \exp\left(-i\frac{\theta}{2}\frac{1}{\sqrt{2}}(\hat{X} + \hat{Z})\right) \\ &= \cos\left(\frac{\theta}{2}\right)\hat{I}_2 - i\sin\left(\frac{\theta}{2}\right)\frac{1}{\sqrt{2}}(\hat{X} + \hat{Z}) \end{aligned}$$

Particularly for $\theta = \pi$, we find

$$\hat{R}_{\hat{\mathbf{n}}}(\pi) = -\frac{i}{\sqrt{2}}(\hat{X} + \hat{Z}) = -i\hat{H} \tag{8}$$

The Hadamard gate transforms the $\hat{Z}$ basis $\{ \mid 0\rangle, |1\rangle \}$ to the $\hat{X}$ basis $\{ \mid +\rangle, |-\rangle \}$:

$$\begin{aligned} \hat{H}\,|0\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \\ \hat{H}\,|1\rangle &= \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle \end{aligned}$$

This transformation creates quantum superposition from the standard computational basis.

In Python, the Hadamard gate can be implemented in the following way using Numpy:

```
import numpy as np
def apply_hadamard(qubit: np.ndarray) -> np.ndarray:
  h = (1/np.sqrt(2)) * np.array([[1., 1.], [1., -1.]], dtype=np.complex128)
  return qubit @ h

# Example
qubit = create_qubit(theta=0, phi=0) # |0> state: [0., 0.]
qubit_h = apply_hadamard(qubit) # Returns |+> state: [0.70710678, 0.70710678]
```

1.1.3 Phase shift-gate

The general phase shift gate is given by

$$\hat{P}(\nu) := |0\rangle\langle 0| + e^{i\nu}|1\rangle\langle 1| = \hat{Z}^{\nu/\pi} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\nu} \end{bmatrix}, \tag{9}$$

where the Pauli Z power gate is defined as

$$\hat{Z}^t := e^{-i\frac{\pi}{2}t(\hat{Z}-\hat{I}_2)} \sim \hat{R}_z(\pi t) \tag{10}$$

which is equivalent to $\hat{R}_z(\pi t)$ up to a global phase. The phase shift gate induces a relative phase to a qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$:

$$\begin{aligned} \hat{P}(\nu)|\psi\rangle =& (|0\rangle\langle 0| + e^{i\nu}|1\rangle\langle 1|)(\alpha|0\rangle + \beta|1\rangle) \\ =& \alpha|0\rangle + e^{i\nu}\beta|1\rangle \end{aligned}$$

For $\nu = \pi/2$, we get the square root of $\hat{Z}$, which is also known as the phase gate, or S gate:

$$\hat{S} = \hat{Z}^{1/2} = \hat{P}(\pi/2) = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \tag{11}$$

The S gate transforms the $\hat{X}$ basis $\{\ |+\rangle, |-\rangle\ \}$ to the basis $\{\ |i\rangle, |-i\rangle\ \}$:

$$\begin{aligned} \hat{S}|+\rangle =& (|0\rangle\langle 0| + i|1\rangle\langle 1|)\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) = |i\rangle \\ \hat{S}|-\rangle =& (|0\rangle\langle 0| + i|1\rangle\langle 1|)\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) = |-i\rangle \end{aligned}$$

In particular, the composition $\hat{S}\hat{H}$ transforms the $\hat{Z}$ basis $\{\ |0\rangle, |1\rangle\ \}$ to the $\{\ |i\rangle, |-i\rangle\ \}$:

$$\begin{aligned} \hat{S}\hat{H}|0\rangle = \hat{S}|+\rangle = |i\rangle \\ \hat{S}\hat{H}|1\rangle = \hat{S}|-\rangle = |-i\rangle \end{aligned}$$

In Python, the S gate can be implemented in the following way using Numpy:

```
import numpy as np
def apply_s_gate(qubit: np.ndarray) -> np.ndarray:
  s_gate = np.array([[1., 0.], [0., 1.j]], dtype=np.complex128)
  return qubit @ s_gate

# Example
qubit = create_qubit(theta=np.pi/2, phi=0) # |+> state: [0.70710678, 0.70710678]
qubit_h = apply_s_gate(qubit) # Returns |i> state: [0.70710678, 0.70710678j]
```

1.2 Bell states

The Bell basis forms an orthonormal basis for the four-dimensional Hilbert space ¶$\mathcal{H}^{\otimes 2}$ and consists of the Bell states

$$
\begin{aligned}
|\Phi_{\pm}\rangle :=& \frac{1}{\sqrt{2}}\left(|00\rangle \pm |11\rangle\right) \\
|\Psi_{\pm}\rangle :=& \frac{1}{\sqrt{2}}\left(|01\rangle \pm |10\rangle\right).
\end{aligned}
$$

The Bell states can be created from the computational basis by applying a Hadamard gate $\hat{H}(0)$ to the first qubit, followed by a controlled NOT gate, $\mathrm{CNOT}(0,1)$:

$$
\begin{aligned}
\mathrm{CNOT}(0,1)\hat{H}(0)\,|00\rangle &= |\Phi_{+}\rangle \\
\mathrm{CNOT}(0,1)\hat{H}(0)\,|01\rangle &= |\Psi_{+}\rangle \\
\mathrm{CNOT}(0,1)\hat{H}(0)\,|10\rangle &= |\Psi_{-}\rangle \\
\mathrm{CNOT}(0,1)\hat{H}(0)\,|11\rangle &= |\Phi_{-}\rangle\,,
\end{aligned}
$$

where

- $\hat{H}(0) = \hat{H} \otimes \hat{I}$, and
- $\mathrm{CNOT}(0,1) = |0\rangle\langle 0| \otimes \hat{I} + |1\rangle\langle 1| \otimes \hat{X}$

A Python function for creating Bell states is shown in Program 2. This function uses methods of the NQubitState class to apply the Hadamard gate and controlled X gate (see Program 11 and Program 13 for their Python implementation).

1.2.1 Binary Qubit Measurement

Program 3 shows Python code for simulating sequential measurements on a binary qubit state using the NQubitState class. The Python implementation for simulating quantum measurements is detailed in Program 12. In this example, the qubit state is initialized to the Bell state $|\Psi_{+}\rangle$, followed by two operations:

1. A Hadamard gate $\hat{H}(1)$ applied to the second qubit
2. A controlled-NOT gate $\mathrm{CNOT}(1,0)$ targeting the second qubit.

The corresponding quantum circuit for this operation is shown in Figure 4. The resulting binary state $|\Lambda\rangle$ can be expressed as follows:

```
def create_bell_state(state: int) -> NQubitState:
  """
  Create a qubit in a Bell state using Hadamard and CNOT gates.

  Args:
    state (int): Bell state to initialize the qubit in. Can be 0, 1, 2, or 3.
    - 0 initializes |00> resulting in | \Phi+> = (1/2)|00> + (1/2)|11>
    - 1 initializes |01> resulting in | \Psi+> = (1/2)|01> + (1/2)|10>
    - 2 initializes |10> resulting in | \Psi -> = (1/2)|00> - (1/2)|11>
    - 3 initializes |11> resulting in | \Phi -> = (1/2)|01> - (1/2)|10>
  """

  bell_state = NQubitState(2, basis_state=state)
  bell_state.hadamard_gate(0)  # Apply Hadamard gate to first qubit
  bell_state.cnot_gate(0, 1)  # Apply CNOT gate

  return bell_state
```

Figure 2: Python for constructing Bell states using the NQubitState class.

$$
\begin{aligned}
|\Lambda\rangle &= \mathrm{CNOT}(1,0)\,\hat{H}(1)\,|\Psi_+\rangle \\
&= \begin{bmatrix} 1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1&1&0&0\\1&-1&0&0\\0&0&1&1\\0&0&1&-1 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 0\\1\\1\\0 \end{bmatrix} \\
&= \frac{1}{2} \begin{bmatrix} 1&1&0&0\\0&0&1&-1\\0&0&1&1\\1&-1&0&0 \end{bmatrix} \begin{bmatrix} 0\\1\\1\\0 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1\\1\\1\\-1 \end{bmatrix} \\
&= \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle).
\end{aligned}
$$

This shows that $|\Lambda\rangle$ is a superposition of the binary computation basis states, with equal magnitude of the probability amplitudes. Specifically, the probability of measuring any of the four computation basis states is

$$
\Pr(|00\rangle) = \Pr(|01\rangle) = \Pr(|10\rangle) = \Pr(|11\rangle) = \left|\frac{1}{2}\right|^2 = \frac{1}{4}. \tag{12}
$$

Thus, we expect the measurement probabilities to be approximately equal for each basis state. Figure 5 shows the results for experimental measurements for $|\Lambda\rangle$ conducted in sequential order with 1000 shots. Thes results correspond to the following observed probabilities

$$
\begin{aligned}
\Pr(|00\rangle) &= \frac{241}{1000} = 0.241 \\
\Pr(|01\rangle) &= \frac{263}{1000} = 0.263 \\
\Pr(|10\rangle) &= \frac{242}{1000} = 0.242 \\
\Pr(|11\rangle) &= \frac{254}{1000} = 0.254
\end{aligned}
$$

The probabilities are relatively close to the theoretical value of 0.25 for each state, as expected.

```
shots = 1000

# Initialize Bell state
selected_state = 1 # | \Psi+>
bell_state = create_bell_state(selected_state)

# Apply Hadamard and CNOT gates
bell_state.hadamard_gate(1)
bell_state.cnot_gate(1,0)

counts = bell_state.measure([0,1], shots)
```

Figure 3: Python code for simulating quantum measurements on a binary qubit state.

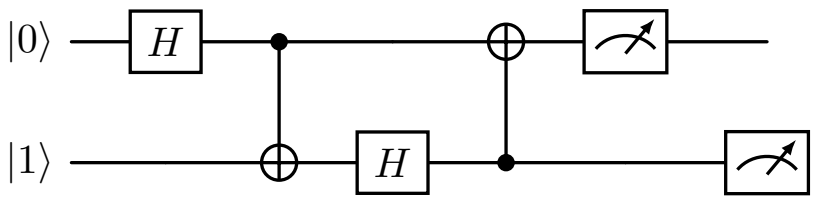

Figure 4: Corresponding quantum circuit for simulating quantum measurements on a binary qubit state.

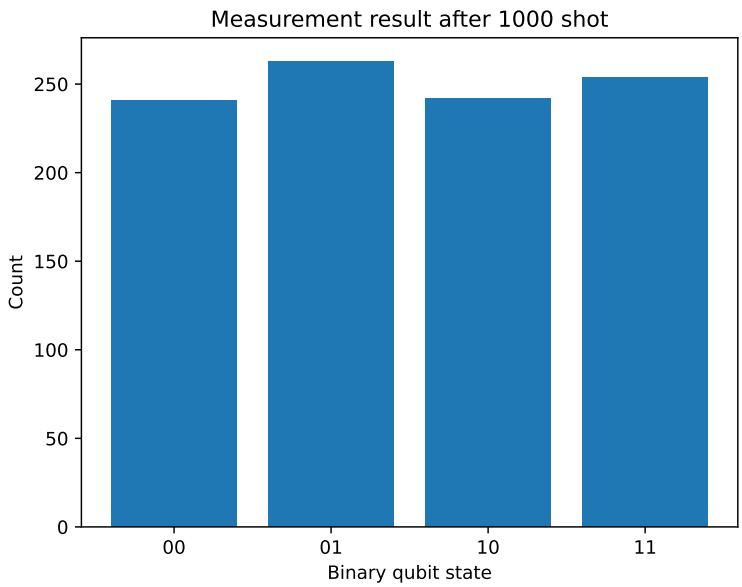

Figure 5: Measurement results for the Bell state $|\Psi_+\rangle$ after applying a Hadamard gate to the first qubit, followed by a CNOT gate, measured over 1000 shots.

2 Part b) One-Qubit Hamiltonian Eigenvalue Problem

In terms of the Pauli basis $\left\{ \hat{I}, \hat{\sigma}_x, \hat{\sigma}_y, \hat{\sigma}_z \right\}$, the total Hamiltonian $\hat{H} = \hat{H}_0 + \hat{H}_{\mathrm{I}}$ can be written

$$\hat{H} = \underbrace{(\mathcal{E} + \lambda c)\hat{I}_2}_{=\hat{H}_0} + \underbrace{(\Omega + \lambda\omega_z)\hat{\sigma}_z + \lambda\omega_x\hat{\sigma}_x}_{=\hat{H}_I} \tag{13}$$

where the parameter are defined as

$$\begin{aligned}
\mathcal{E} =& \frac{E_1 + E_2}{2} \\
\Omega =& \frac{E_1 - E_2}{2} \\
c =& \frac{V_{11} + V_{22}}{2} \\
\omega_z =& \frac{V_{11} - V_{22}}{2} \\
\omega_x =& V_{12} = V_{21}.
\end{aligned}$$

To find the eigenvalues of $\hat{H}$, we must solve the characteristic equation $\det(\hat{H} - E\hat{I}) = 0$. The identity matrix $\hat{I}$ contributes a constant energy shift $\mathcal{E} + \lambda c$, meaning the relevant part of the Hamiltonian for eigenvalue calculation is

$$\hat{H}' = (\Omega + \lambda\omega_z)\sigma_z + \lambda\omega_x\sigma_x, \tag{14}$$

or in matrix form

$$\hat{H}' = \begin{bmatrix} \Omega + \lambda\omega_z & \lambda\omega_x \\ \lambda\omega_x & -(\Omega + \lambda\omega_z) \end{bmatrix} \tag{15}$$

To find the eigenvalues, we solve the characteristic equation $\det(\hat{H}' - E'\hat{I}) = 0$:

$$\begin{aligned}
0 =& \begin{vmatrix} \Omega + \lambda\omega_z - E' & \lambda\omega_x \\ \lambda\omega_x & -(\Omega + \lambda\omega_z) - E' \end{vmatrix} \\
=& (\Omega + \lambda\omega_z - E')(-(\Omega + \lambda\omega_z) - E') - (\lambda\omega_x)^2 \\
=& -(\Omega + \lambda\omega_z - E')(\Omega + \lambda\omega_z + E') - (\lambda\omega_x)^2 \\
=& -(\Omega + \lambda\omega_z)^2 + E'^2 - (\lambda\omega_x)^2
\end{aligned}$$

Rearranging yields

$$E'^2 = (\Omega + \lambda\omega_z)^2 + (\lambda\omega_x)^2. \tag{16}$$

Taking the square root, we find the reduced eigenvalues

$$E'_\pm = \pm\sqrt{(\Omega + \lambda\omega_z)^2 + (\lambda\omega_x)^2} \tag{17}$$

Adding the constant shift $\mathcal{E} + \lambda c$, we finally arrive at

$$E_{\pm} = \mathcal{E} + \lambda c \pm \sqrt{(\Omega + \lambda\omega_z)^2 + (\lambda\omega_x)^2} \tag{18}$$

The Python code for computing the eigenvalues of $\hat{H}$ is provided inProgram 15. The resulting eigenvalues as a function of the interaction strength $\lambda \in [0, 1]$ are plotted in Figure 6, using the following parameters:

- $E_1 = 0$,
- $E_2 = 4$
- $V_{11} = -V_{22} = 3$
- $V_12 = V_21 = 0.2$

The eigenvalue structure of $\hat{H}$ exhibits the characteristic behavior of an avoided crossing, a common feature in two-level quantum systems. In the non-interacting limit $\lambda = 0$, the Hamiltonian reduces to the diagonal form $\hat{H} = \hat{H}_0$,corresponding to two uncoupled states. In this regime, the system remains in the computational basis, with eigenstates $|0\rangle$ and $|1\rangle$ having eigenvalues $E_1 = 0$ and $E_2 = 4$, respectively.

As the interaction strength λ increases, the off-diagonal coupling terms $V_{12} = V_{21}$ induce mixing between the computational basis states. Around $\lambda = 2/3$, the eigenstates become maximally mixed and they swap their dominant character: the lower eigenstate transition from being predominantly $|0\rangle$ to primarily $|1\rangle$, while the higher eigenstate undergoes the reverse transition. Beyond this avoided crossing point, $\lambda > 2/3$, the eigenstates continue to separate in energy, with their dominant character exchanged.

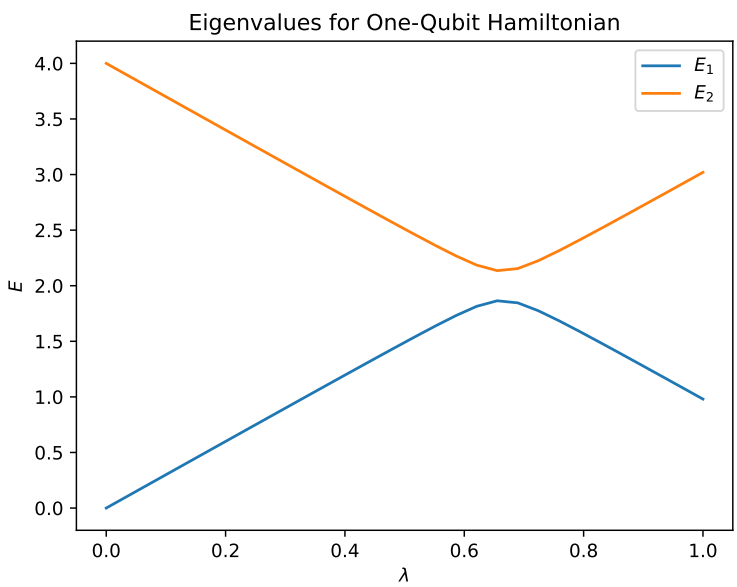

Figure 6: Exact eigenvalues of the one-qubit Hamiltonian as a function of the interaction strength λ.

3 Part c) VQE for One-Qubit Hamiltonian

In terms of a qubit state $|psi\rangle$, the expectation value for the Hamiltonian (13) is given by

$$\langle H\rangle_\psi = \langle\psi|\hat{H}|\psi\rangle = (\mathcal{E}+\lambda c)\underbrace{\langle\psi|\hat{I}_2|\psi\rangle}_{=1} + (\Omega+\lambda\omega_z)\langle\psi|\hat{Z}|\psi\rangle + \lambda\omega_x\langle\hat{X}\rangle$$

To compute the expectation value $\langle H\rangle_\psi$, a quantum circuit must be configured with two measurement bases: one for the $\hat{Z}$-term and another for the $\hat{X}$-term. The expectation value of $\hat{Z}$ is measured directly in the computational basis, while the $\hat{X}$-term requires a basis transformation via a Hadamard gate before measurement. To implement the variational quantum eigensolver (VQE), each qubit is prepared in an ansatz state $|\psi(\theta,\phi)\rangle$, parametrized by θ and ϕ using rotations gates:

$$|\psi(\theta,\phi)\rangle = \hat{R}_y(\phi)\hat{R}_x(\theta)\,|0\rangle \tag{19}$$

These variational parameters are itertively optimized using classical algorithms to minimize $\langle H\rangle$, approximating the ground-state energy of $\hat{H}$. Figure 7 illustrates the quantum circuit for this VQE setup. The Python function for computing $\langle H\rangle$ using quantum circuit logic is given in Program 22, while the function for minimizing $\langle H\rangle$ using gradient descent is provided in Program 19.

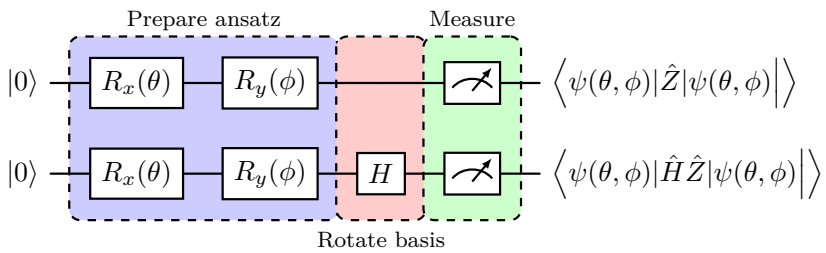

Figure 7: Quantum circuit for the variational quantum eigensolver (VQE) applied to the one-qubit Hamiltonian with a \hat{Z}-term and $\hat{X} - term$.

Figure 8 shows the VQE energies for the one-qubit Hamiltonian optimized using gradient descent with Adaptive Moment Estimation (ADAM) over 500 epochs. The expectation values $\langle H\rangle$ were computed from quantum circuit measurements over 1000 shots. The estimated energies closely align with the analytical values, demonstrating the effectiveness of VQE in approximating ground-state energies. However, repeated runs sometimes yield deviations, likely due to sub-optimal random initialization of variational parameters. This can be solved by implementing a more informed method of initalizing the variational parameters.

4 Part d) Two-Qubit Hamiltonian Eigenvalue Problem

The Python code for computing the eigenvalues of the two-qubit Hamiltonian is given in

$H_x = 2.0,\ H_z = 3.0$

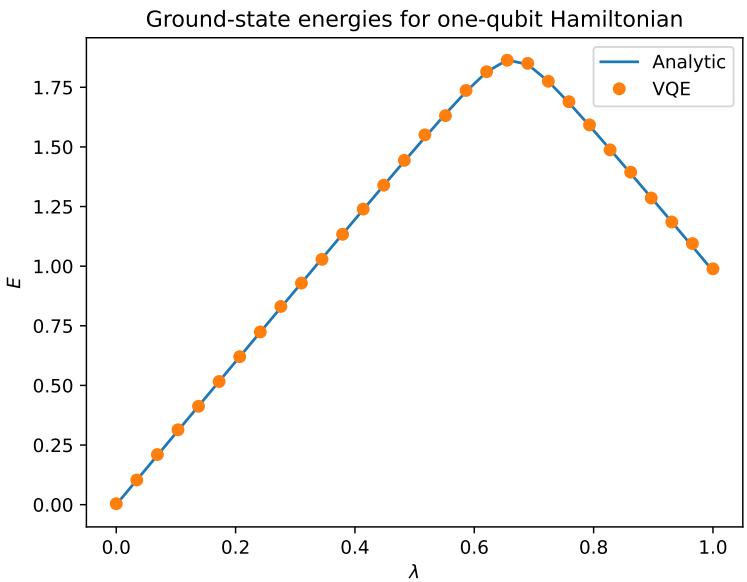

Figure 8: A plot of VQE energies of a one-qubit Hamiltonian, optimized using gradient descent with Adaptive Moment Estimation (ADAM).

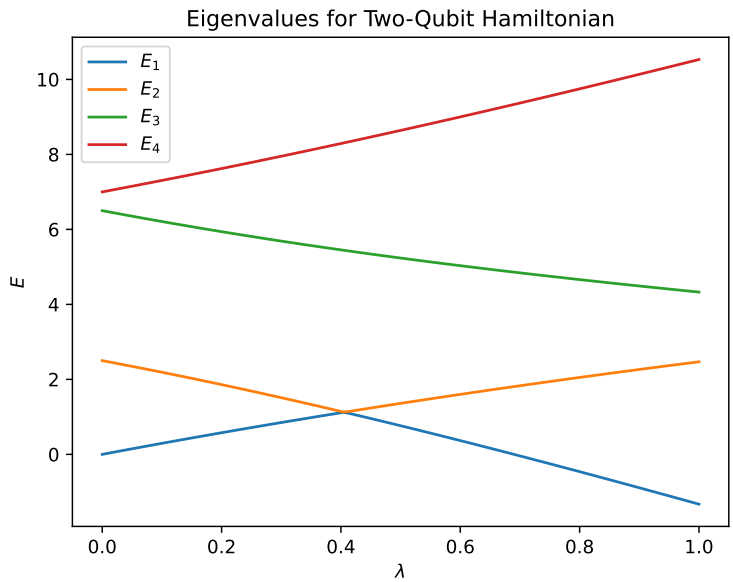

Figure 9: Exact eigenvalues of the two-qubit Hamiltonian as a function of the interaction strength λ.

5 Appendix

5.1 Code Repository

The Python source code used for this project is available at https://github.com/semapheur/fys5419.

5.2 Source Code

5.2.1 Qubit Class

5.2.2 One-Qubit Hamiltonian Energies

5.2.3 Two-Qubit Hamiltonian Energies and Entropies

5.2.4 VQE Ansatz

5.2.5 Parameter Shift Gradient

5.2.6 VQE Gradient Descent (ADAM)

5.2.7 One-Qubit Hamiltonian Quantum Circuit Computation

```
import numpy as np

class NQubitState:
  def __init__(
    self,
    num_qubits: int,
    state: NDArray[np.complex128] | None = None,
    basis_state: int | None = 0,
  ):
    self.num_qubits = num_qubits
    self.dim = 2**num_qubits

    if state is not None:
      if state.shape != (self.dim,):
        raise ValueError(f"State vector must be {self.dim} -dimensional")
      self._validate_and_set_state(state)
      return

    # Initialize to specified basis state (default |0...0)
    state = np.zeros(self.dim, dtype=np.complex128)
    state[basis_state] = 1.0
    self._state = state

  def _validate_and_set_state(self, state: NDArray[np.complex128]):
    # Ensure complex type
    state = state.astype(np.complex128)

    # Check normalization
    norm = np.linalg.norm(state)
    if not np.isclose(norm, 1.0, rtol=1e -6):
      raise ValueError(f"State vector not normalized. Norm: {norm}")

    self._state = state

  @property
  def state(self) -> NDArray[np.complex128]:
    return self._state

  @state.setter
  def state(self, state: NDArray[np.complex128]):
    self._validate_and_set_state(state)
```

Figure 10: Class constructer for an n-ary qubit state implemented in Python.

```python
class NQubitState:
  ...

  def apply_unary_gate(self, gate: NDArray[np.complex128], target: int):
    """Apply a unary quantum gate to the specified qubit.

    Args:
      gate (NDArray[np.complex128]): 2x2 unitary matrix representing the gate
      target (int): Index of target qubit"""

    if gate.shape != (2, 2):
      raise ValueError("Gate must be a 2x2 matrix")

    # Construct full gate matrix using tensor products
    full_gate = np.array([[1]], dtype=np.complex128)
    for i in range(self.num_qubits):
      full_gate = np.kron(full_gate, gate if i == target else np.eye(2)).astype(
        np.complex128
      )

    self.state = full_gate @ self.state

  def apply_controlled_gate(
    self, gate: NDArray[np.complex128], control: int, target: int
  ):
    """Apply a controlled single -qubit gate.

    Args:
      gate (NDArray[np.complex128]): 2x2 unitary matrix representing the gate
      control (int): Index of control qubit
      target (int): Index of target qubit"""

    if gate.shape != (2, 2):
      raise ValueError("Gate must be a 2x2 matrix")

    # Initialize controlled gate
    controlled_gate = np.eye(self.dim, dtype=np.complex128)

    for i in range(self.dim):
      if (i >> target) & 1:  # Apply gate if target qubit is 1
        control_bit = (i >> control) & 1  # Get bit value of control qubit
        i_modified = i ^ (1 << control)  # Index of flipped control qubit

        controlled_gate[i, i] = gate[control_bit, control_bit]
        controlled_gate[i, i_modified] = gate[control_bit, 1 - control_bit]

    self.state = controlled_gate @ self.state
```

Figure 11: Methods for applying unary and controlled quantum gates in the the NQubitState class implemented in Python.

```python
class NQubitState:
  ...

  def measure(
    self, targets: list[int], shots: int = 1, as_dict: bool = True
  ) -> dict[str, int] | list[tuple[int, ...]]:
    """Perform sequential measurements on specified qubits.

    Args:
      target (list[int]): List of qubit indices to measure in sequence
      shots (int): Number of measurements to perform

    Returns:
      dict[str, int] | list[tuple[int, ...]]: Measurement outcomes
    """

    # Generate measurement probabilities for each shot
    random_outcomes = np.random.random(size=(shots, len(targets)))

    outcomes = _measure_shots(self.state.copy(), targets, random_outcomes, self.dim)

    result = [tuple(outcome) for outcome in outcomes]
    if not as_dict:
      return result

    outcome_strings = ["".join(map(str, outcome)) for outcome in result]
    return dict(sorted(Counter(outcome_strings).items()))
```

Figure 12: A method for performing qubit measurements within the NQubitState class implemented in Python.

```
class NQubitClass:
  ...
  def hadamard_gate(self, target: int):
    """Apply a Hadamard gate to the specified qubit.

    Args:
      target (int): Index of target qubit"""

    H = np.array([[1, 1], [1, -1]], dtype=np.complex128) / np.sqrt(2)
    self.apply_unary_gate(H, target)

  def s_gate(self, target: int):
    """Apply the S gate to the specified qubit.

    Args:
      target (int): Index of target qubit"""

    s = np.array([[1, 0], [0, 1j]], dtype=np.complex128)
    self.apply_unary_gate(s, target)

  def s_gate_dagger(self, target: int):
    """Apply the S gate to the specified qubit.

    Args:
      target (int): Index of target qubit"""

    s_dagger = np.array([[1, 0], [0, -1j]], dtype=np.complex128)
    self.apply_unary_gate(s_dagger, target)

  def cnot_gate(self, control: int, target: int):
    """
    Apply the controlled -NOT (CNOT) gate with specified control qubit.

    Args:
      control (int): Index of control qubit
      target (int): Index of target qubit"""

    x = np.array([[0, 1], [1, 0]], dtype=np.complex128)
    self.apply_controlled_gate(x, control, target)
```

Figure 13: A selection quantum gate methods for the NQubitState class implemented in Python.

```
from numba import jit
import numpy as np

@jit(nopython=True)
def _measure_shots(
  state: NDArray[np.complex128],
  targets: list[int],
  random_outcomes: np.ndarray,
  dim: int,
) -> np.ndarray:

  shots = random_outcomes.shape[0]
  num_targets = len(targets)

  outcomes = np.zeros((shots, num_targets), dtype=np.int32)

  for shot in range(shots):
    current_state = state.copy()

    for i, target in enumerate(targets):
      # Bit mask for target qubit
      mask = 1 << target

      # Calculate probability for measuring |0 on target qubit
      prob_0 = 0.0
      for j in range(dim):
        if (j & mask) == 0:  # If target qubit is |0
          prob_0 += np.abs(current_state[j]) ** 2

      # Measure target qubit
      outcome = int(random_outcomes[shot, i] > prob_0)
      outcomes[shot, i] = outcome

      # Collapse state
      norm_squared = 0.0
      for j in range(dim):
        bit_value = (j & mask) >> target
        if bit_value != outcome:
          current_state[j] = 0.0
        else:
          norm_squared += np.abs(current_state[j]) ** 2

      # Normalize state
      if norm_squared > 1e -10:
        norm = np.sqrt(norm_squared)
        for j in range(dim):
          current_state[j] /= norm

  return outcomes
```

Figure 14: A Python function for simulating qubit state measurements, optimized using Numba's JIT compiler for improved performance

```python
import numpy as np

def analytic_energies(lambdas: np.ndarray) -> np.ndarray:
  """
  Calculate the analytic energies of a single -qubit Hamiltonian
  for a set of interaction strengths.

  Args:
    lambdas (np.ndarray): Array of interaction strengths

  Returns:
    np.ndarray: 2 -column array of analytic energies,
    sorted in ascending order for each interaction strength
  """

  root = np.sqrt(np.square(omega + lambdas * omega_z) + np.square(lambdas * omega_x))

  E_minus = epsilon + lambdas * c - root
  E_plus = epsilon + lambdas * c + root

  return np.column_stack((E_minus, E_plus))

def numeric_energies(lambdas: np.ndarray) -> np.ndarray:
  """
  Calculate the exact energies of a single -qubit Hamiltonian
  for a set of interaction strengths.

  Args:
    lambdas (np.ndarray): Array of interaction strengths

  Returns:
    np.ndarray: 2 -column array of exact energies,
    sorted in ascending order for each interaction strength
  """

  return np.array([np.linalg.eigvalsh(hamiltonian(lmb)) for lmb in lambdas])
```

Figure 15: Python functions for computing the eigenvalues of the one-qubit Hamiltonian.

```
import numpy as np

sigma_x = np.array([[0.0, 1.0], [1.0, 0.0]])
sigma_z = np.array([[1.0, 0.0], [0.0, -1.0]])
I2 = np.eye(2)

H_x = 2.0
H_z = 3.0

eps_00 = 0.0
eps_01 = 2.5
eps_10 = 6.5
eps_11 = 7.0

def hamiltonian(interaction_strength: float):
  """Construct the Hamiltonian matrix for a two -qubit quantum system."""
  H_I = H_z * np.kron(sigma_z, sigma_z) + H_x * np.kron(sigma_x, sigma_x)
  H_0 = np.diag([eps_00, eps_01, eps_10, eps_11])
  H = H_0 + interaction_strength * H_I

  return H

def exact_energies_and_entropies(lambdas: NDArray[np.float64]):
  """
  Calculate the exact energies and von Neumann entropies for a two -qubit system Hamiltonian

  Args:
    lambdas (NDArray[np.float64]): Array of interaction strengths.

  Returns:
    Tuple[NDArray[np.float64], NDArray[np.float64]]:
      - A 2D array of exact energies, with each row corresponding to a lambda value.
      - A 2D array of von Neumann entropies, with each row corresponding to a lambda value.
  """

  energies = np.zeros((len(lambdas), 4))
  entropies = np.zeros((len(lambdas), 4))

  for i, lmb in enumerate(lambdas):
    H = hamiltonian(lmb)
    eigenvalues, eigenvectors = np.linalg.eig(H)

    permute = eigenvalues.argsort()
    energies[i] = eigenvalues[permute]
    eigenvectors = eigenvectors[:, permute]

    # Calculate von Neumann entropy
    for j in range(4):
      sub_density = trace_out(eigenvectors[:, j], 0)
      eigenvals_density = np.linalg.eigvalsh(sub_density)
      eigenvals_density = np.ma.masked_equal(eigenvals_density, 0.0).compressed()
      entropies[i, j] = -np.sum(eigenvals_density * np.log2(eigenvals_density))

  return energies, entropies
```


Figure 16: Python functions for computing the eigenvalues of the two-qubit Hamiltonian, and the von Neumann entropies for the corresponding eigenstates.

```python
def prepare_ansatz(angles: np.ndarray) -> NQubitState:
  """
  Prepare a VQE ansatz state for an arbitrary number of qubits
  using rotation X and Y gates.

  Args:
    angles (np.ndarray): Parameter vector
      [theta_0, phi_0, theta_1, phi_1, ..., theta_n, phi_n]
    num_qubits (int): Number of qubits in the ansatz state

  Returns:
    NQubitState: VQE ansatz state
  """

  if len(angles) % 2 != 0:
    raise ValueError("Number of angles must be even")

  num_qubits = len(angles) // 2

  # Initialize the ansatz state
  qubit = NQubitState(num_qubits)

  # Apply rotation X and Y gates
  for i in range(num_qubits):
    theta_i = angles[2 * i]
    phi_i = angles[2 * i + 1]
    qubit.rotation_x_gate(theta_i, i)
    qubit.rotation_y_gate(phi_i, i)

  if num_qubits == 1:  # Special case for single qubit
    return qubit

  # Apply CNOT gates to entangle adjacent qubits
  for i in range(num_qubits - 1):
    qubit.cnot_gate(i, i + 1)

  return qubit
```

Figure 17: Python function for preparing a VQE ansatz state using $\hat{R}_x$ and $\hat{R}_y$ rotation gates.

```python
from concurrent.future import ThreadPoolExecutor
import numpy as np

def parameter_shift_gradient(
  angles: np.ndarray, interaction_strength: float, energy_fn: Callable
) -> np.ndarray:
  """
  Calculate gradient using parameter shift rule.

  Args:
    angles (np.ndarray): Parameter vector [theta, phi]
    interaction_strength (float): Interaction strength parameter
    energy_fn (Callable): Energy calculation function

  Returns:
    np.ndarray: Gradient vector
  """

  grad = np.zeros_like(angles)
  shift = np.pi / 2

  shifts: list[tuple[np.ndarray, float]] = []
  for i in range(len(angles)):
    angles_plus = angles.copy()
    angles_plus[i] += shift
    shifts.append((angles_plus, interaction_strength))

    angles_minus = angles.copy()
    angles_minus[i] -= shift
    shifts.append((angles_minus, interaction_strength))

  with ThreadPoolExecutor() as executor: # Parallelize
    energies = list(executor.map(lambda args: energy_fn(*args), shifts))

  for i, (energy_plus, energy_minus) in enumerate(zip(energies[::2], energies[1::2])):
    grad[i] = (energy_plus - energy_minus) / (2 * shift)

  return grad
```

Figure 18: Python function for calculating the gradient of energy expectation using parameter shift.

```python
import numpy as np

def minimize_energy(
  angles_0: np.ndarray,
  energy_fn: Callable[[np.ndarray, float], float],
  lmb: float,
  learning_rate: float = 0.01,
  epochs: int = 1000,
  tolerance: float = 1e -6,
) -> tuple[np.ndarray, float, float, int]:

  angles = angles_0.copy()
  epoch = 0
  delta_energy = float("inf")
  min_energy = float("inf")
  no_improvement_epochs = 0

  energy = energy_fn(angles, lmb)
  energies = [energy]
  angle_store = [angles.copy()]

  dim = angles.shape[0]
  optimizer_state = {"momentums": np.zeros(dim), "velocities": np.zeros(dim)}

  while epoch < epochs and delta_energy > tolerance:
    # Calculate gradient
    grad = parameter_shift_gradient(angles, lmb, energy_fn)

    # Update learning rate and optimizer state
    update, optimizer_state = optimizer_update(
      method, grad, optimizer_state, learning_rate, epoch, **OPTIMIZER_PARAMS[method]
    )
    angles += update

    # Update energy and delta_energy
    energy_new = energy_fn(angles, lmb)
    delta_energy = np.abs(energy_new - energy)
    energy = energy_new

    epoch += 1

  final_energy = energy_fn(angles, lmb)
  return angles, energy, delta_energy, epoch
```

Figure 19: Python function for minimizing the energy expectation using gradient descent.

```
OPTIMIZER_PARAMS = {
  "adam": {"beta1": 0.9, "beta2": 0.999, "epsilon": 1e -8},
  "rmsprop": {"decay_rate": 0.9, "epsilon": 1e -8},
  "adagrad": {"epsilon": 1e -8},
  "sgd": {"momentum": 0.9},
}

class OptimizerState(TypedDict, total=False):
  squared_grad: np.ndarray | None
  grad_accumulator: np.ndarray | None
  momentum: float | None
  momentums: np.ndarray | None
  velocities: np.ndarray | None

def optimizer_update(
  method: str,
  gradient: np.ndarray,
  state: OptimizerState,
  learning_rate: float,
  epoch: int,
  **kwargs,
):
  if method == "adam":
    beta1 = cast(float, kwargs.get("beta1", 0.9))
    beta2 = cast(float, kwargs.get("beta2", 0.999))
    epsilon = cast(float, kwargs.get("epsilon", 1e -8))

    momentums = cast(np.ndarray, state["momentums"])
    velocities = cast(np.ndarray, state["velocities"])

    momentums = beta1 * momentums + (1 - beta1) * gradient
    velocities = beta2 * velocities + (1 - beta2) * np.square(gradient)

    momentums_hat = momentums / (1 - beta1 ** (epoch + 1))
    velocities_hat = velocities / (1 - beta2 ** (epoch + 1))

    update = -learning_rate * momentums_hat / (np.sqrt(velocities_hat) + epsilon)
    state["momentums"] = momentums
    state["velocities"] = velocities

  ...
  else:  # Default to gradient descent
    update = -learning_rate * gradient

  return update, state
```

Figure 20: Python function for updating the optimizer state in gradient descent. Only the implementation for the Adaptive Moment Estimation (ADAM) method is shown here. The full implementation is available in the code repository for this project.

```python
def vqe_energies(
  angle_parameters: int,
  energy_fn: Callable,
  lambdas: np.ndarray,
  shots: int,
  max_epochs: int,
  learning_rate: float,
  method: Optimizer,
  verbose: bool = False,
):
  partial_energy_fn = partial(energy_fn, shots=shots)
  vqe_energies = np.zeros(len(lambdas))
  epochs = np.zeros(len(lambdas))
  optimal_angles = []

  for i, lmb in enumerate(lambdas):
    if verbose:
      print(f"\nProcessing lambda = {lmb:.4f} ({i + 1}/{len(lambdas)})")

    best_energy = float("inf")
    best_angles = None
    best_epoch = 0

    angles_0 = np.random.uniform(0, np.pi, angle_parameters)
    angles, energy, _, epoch = minimize_energy(
      angles_0,
      partial_energy_fn,
      lmb,
      learning_rate=learning_rate,
      epochs=max_epochs,
      method=method,
      verbose=verbose,
    )

    if energy < best_energy:
      best_energy = energy
      best_angles = angles
      best_epoch = epoch

    vqe_energies[i] = best_energy
    epochs[i] = best_epoch
    optimal_angles.append(best_angles)

  return vqe_energies, epochs, optimal_angles
```

Figure 21: Python function for computing the VQE energies of a Hamiltonian as a function of interaction strength.

```python
def energy_expectation(
  angles: np.ndarray, interaction_strength: float, shots: int
) -> float:
  """
  Calculate energy expectation value for a single -qubit Hamiltonian of the form
  H = epsilon * I + omega * Z + c * I + omega_z * Z + omega_x * X

  Args:
    angles (np.ndarray): Parameter vector [theta, phi]
    interaction_strength (float): Interaction strength parameter
    shots (int): Number of measurement shots

  Returns:
    float: Energy expectation value
  """

  # Prepare ansatz
  ansatz = prepare_ansatz(angles)

  # Measure in Z -basis
  qubit = ansatz.copy()
  outcomes = cast(dict[str, int], qubit.measure([0], shots))
  measure_z = measurement_expectation(outcomes, shots)

  # Measure in X -basis
  qubit = ansatz.copy()
  qubit.hadamard_gate(0)  # Rotate to X -basis
  outcomes = cast(dict[str, int], qubit.measure([0], shots))
  measure_x = measurement_expectation(outcomes, shots)

  # Calculate expectation value
  exp_val_z = (omega + interaction_strength * omega_z) * measure_z
  exp_val_x = interaction_strength * omega_x * measure_x
  exp_val_i = epsilon + c * interaction_strength
  exp_val = exp_val_z + exp_val_x + exp_val_i
  return exp_val
```

Figure 22: Python function for computing the expectation of a one-qubit Hamiltonian using quantum circuit logic.